AI Applications in Research

Core Concepts and the Use of Claude Code in Empirical Work

Prepared from *Scott's Mixtape* (Scott Cunningham) and related discussion

August 2, 2026

Abstract

This document summarises the main concepts surrounding the use of AI *agents*—coding agents such as Claude Code in particular—in academic research, with an emphasis on empirical social science. It draws on the ongoing series in Scott Cunningham's *Scott's Mixtape* newsletter, which has become one of the more sustained public treatments of what it means to do research when the marginal cost of producing cognitive output has collapsed. The core argument is that AI agents dramatically lower the cost of *producing* analytical output, while leaving the cost of *verifying* that output largely unchanged—creating both an opportunity and a new class of risks for the scientific record.

Contents

1 What a Coding Agent Is **1**

2 The Central Idea: Falling Marginal Cost of Production **1**

3 Production Is Not Verification **2**

4 Aggregate Errors and the Scientific Record **2**

5 Practical Applications in the Research Workflow **3**

6 Discipline: Workflow, Reproducibility, and the "Canon" **3**

7 Summary of Key Takeaways **4**

Primary Source **4**

1 What a Coding Agent Is

A **coding agent** is a large-language-model system that can read a project, plan a task, run commands, edit files, execute code, inspect the results, and iterate—acting on a codebase rather than merely emitting text. **Claude Code** (Anthropic's command-line agent) and Codex are the canonical examples. For an empirical researcher, the agent operates directly on the working directory: it can clean data, write and run estimation scripts, build tables and figures, and manage the pipeline that ties them together.

Two properties distinguish an agent from an autocomplete assistant:

- **Tool use.** The agent executes real commands (shell, Stata, R, Python), reads their output, and reacts—closing the loop between writing code and observing its behaviour.
- **Persistence and context.** The agent holds a working memory of the project, follows multi-step plans, and can be steered by project-level instruction files and connected services (e.g. via the Model Context Protocol, MCP).

2 The Central Idea: Falling Marginal Cost of Production

Cunningham frames research output as the result of a *production function* for "cognitive output." A single unit of output (a cleaned dataset, a table, a robustness check) can be produced with different ratios of *machine time* M to *human time* H.

Before agents. Production was *human-time intensive*. Machine time was little more than the seconds Stata needed to invert a matrix; almost all of the cost was the researcher's own hours. The isoquants of the production function were steep and concave.

After agents. The isoquants straighten toward *perfect substitutes*: machine time and human time become close substitutes in producing a unit of output. A cost-minimising researcher then shifts the mix as far toward M (machine time) as possible—delegating production to the agent. Formally, if M and H are near-perfect substitutes,

$$Q \;=\; \alpha\, M + \beta\, H,$$

then the cost-minimising input bundle for a target level of output moves to the low-cost corner, and with cheap machine time that corner is M-intensive.

Consequence. The marginal cost of producing an additional analysis falls sharply. By mid-2026 many researchers report that agents have *dramatically reduced the marginal cost of producing* empirical results, changing what a single researcher can attempt.

3 Production Is Not Verification

The pivotal concept in Cunningham's treatment is the **cleavage between production and verification**. The *production function* is not the *verification function*.

- AI agents collapse the cost of **producing** cognitive output.
- They do *not* collapse the cost of **verifying** that the output is correct—or even that it is what the researcher intended to build in the first place.

This opens a **wedge**: output accumulates faster than the researcher's ability to check it. The disorienting part is not only "is this correct?" but "is this even the thing I meant to make?" Verification—judgement, domain knowledge, knowing what a credible result looks like—becomes the binding constraint on good research.

Managers versus individual contributors. Cunningham draws an analogy to the two archetypes of the tech economist: the *manager* and the *individual contributor*. Working with an agent recasts the researcher as a manager of a tireless research assistant—closer to how a large lab (his example is Raj Chetty's Opportunity Insights) already operates, where the principal investigator directs and verifies rather than writing every line of code. The human's comparative advantage migrates from *writing* to *specifying, directing, and checking*.

4 Aggregate Errors and the Scientific Record

If production is cheap and verification is not, the aggregate risk is more *unverified* output entering the literature. Cunningham motivates this with a personal story: a classic Stata coding error—stray, stale code from a cleaning file—discovered only at the revise-and-resubmit stage, which turned a significant, interesting result into a null one. He never resubmitted the paper.

His estimate is sobering: for every such error that is *caught* and quietly withdrawn, many more go uncaught, are resubmitted, and are ultimately published. This yields the two-sided ledger for "Claude-Code-for-science":

- **Pro.** If agents can find these errors, we avoid a bad counterfactual—fewer false results enter the scientific record. In aggregate this seems clearly desirable.
- **Con (the human-capital question).** Mistakes teach. The coding error taught him organised folders, separation of raw and cleaned data, automated pipelines, version control, and—above all—scientific integrity and a wariness of "liking one's results." If agents remove the ability to make mistakes, does the researcher still accumulate that hard-won human capital? Like playing poker for real versus pretend stakes, *stakes change behaviour and stakes are how you learn*.

The unresolved tension: we want accuracy *now* (wear the seatbelt), but scientific progress also depends on the human capital that failure produces. The document offers this as a genuine open question, not a settled answer.

5 Practical Applications in the Research Workflow

Concretely, coding agents are used across the empirical pipeline:

Data cleaning and construction
Transforming raw data into analysis samples, with the agent documenting each step.

Estimation
Writing and running models (e.g. difference-in-differences, event studies) and iterating on specifications.

Tables and figures
Generating publication-ready output automatically from code, rather than by hand.

Pipeline automation
Wiring cleaning → estimation → output into a single reproducible run.

Error detection
Auditing existing code for the "classic" mistakes (stale variables, sample-size drift, deprecated syntax).

Writing and exposition
Drafting, restructuring, and—via MCP integrations—connecting the agent to external tools and publishing platforms (Cunningham notes experimenting with a Substack–Claude Code MCP integration).

6 Discipline: Workflow, Reproducibility, and the "Canon"

Because agents amplify both good and bad practice, they raise the return to **structured workflows**. Cunningham's warning case is watching an analytical sample silently move from N to $N + k$ observations mid-project—a signal that something upstream changed without being noticed. The remedies are the classic reproducibility disciplines, now more important, not less:

- Separate **raw** data from **cleaned** data; never edit raw data in place.
- Run everything through an **automated pipeline** so tables and figures regenerate deterministically.
- Use **version control** so every change is attributable and reversible.
- Guard against **stale code and deprecation**—the "canon" of a project must be kept current, and dead code pruned.
- Treat the agent's output as a draft to be **verified**, keeping a human in the loop for judgement and integrity.

A related caution comes from Paul Goldsmith-Pinkham's NBER talk on AI in empirical research, which Cunningham highlights: *"prose is not constraints."* A natural-language instruction is not the same as a formal, checkable specification, and researchers must not mistake fluent description for verified correctness.

7 Summary of Key Takeaways

1. AI coding agents (Claude Code, Codex) act on a codebase—running, editing, and iterating—not just suggesting text.
2. They collapse the **marginal cost of producing** empirical output; machine time and human time become near-perfect substitutes.
3. **Production is not verification**: checking correctness and intent is now the binding constraint.
4. The researcher's role shifts from writing code toward **managing and verifying** an AI research assistant.
5. Cheap production risks **aggregate errors** in the scientific record unless verification keeps pace.
6. Mistakes have historically built researcher **human capital**; a world with fewer mistakes poses a genuine, unresolved trade-off.
7. **Reproducibility discipline**—raw/clean separation, pipelines, version control, pruning stale code—matters more, not less.
8. "Prose is not constraints": fluent instructions are not verified specifications.

Primary Source

Scott Cunningham, *Scott's Mixtape* newsletter, https://causalinf.substack.com/ — in particular the ongoing "Claude Code" essay series (2026), including "Claude Code 56: Falling marginal costs of production, output, and aggregate errors," "Stale code, deprecation, and the canon," and "Which research mistakes matter for human capital?"